

Implementación Front-End, Renderizado Dinámico y Mini CRUD para el Portal Tecnológico SYNAPSE.TECH

Transición de Prototipos de Alta Fidelidad a Código Real utilizando HTML5 Semántico, CSS3 Vanilla, Bootstrap 5.3 y JavaScript Nativo con Persistencia en el Cliente

Integrantes — Grupo 26:

Cristian Camilo Ricardo Díaz (Líder / Frontend Developer)

Juan Esteban Serna Serna (Frontend Developer)

Juan Manuel Saldarriaga Arboleda (Frontend Developer)

Módulo: Front-End (2026-I)

Tutor: Jhon Olarte

Institución: Politécnico Grancolombiano

Facultad: Ingeniería, Diseño e Innovación

Fecha: 25 de septiembre de 2026

Repositorio Oficial GitHub:

<https://github.com/cr1c4rd0/Frontend-GRUPO26-POLI>

Contenido

1. Introducción	3
2. Objetivos del Proyecto (General y Específicos)	3
3. Estructura y Autonomía de Ejecución Local	4
4. Matriz Comparativa: Prototipos (Entrega 1) vs. Desarrollo Real (Entrega 2)	5
4.1. Página de Inicio (Home) y Catálogo Destacado	5
4.2. Vista Detallada de Noticia (Article Detail)	6
4.3. Biblioteca de Noticias Favoritas con Persistencia	7
4.4. Formulario de Contacto y Feedback de Validación / Envío	8
4.5. Panel de Administración y Mini CRUD (Creación, Edición y Borrado)	9
4.6. Perfil de Usuario y Preferencias	10
5. Arquitectura Técnica y Solución de Requisitos Funcionales	11
5.1. Renderizado Dinámico desde JSON y Resiliencia CORS	11
5.2. Persistencia en el Cliente mediante Web Storage (localStorage)	12
5.3. Validaciones de Formulario en Tiempo Real con JavaScript	13
5.4. Sistema de Diseño CSS3 y Bootstrap 5.3 (Tokens y Dark Theme)	14
6. Enlace al Repositorio de Código Fuente (GitHub)	15
7. Conclusiones y Lecciones Aprendidas	15
8. Referencias Bibliográficas (Normas APA 7ma Edición)	16

1. Introducción

El presente documento constituye el informe técnico de la **Entrega 2** del módulo Front-End (Semana 5), correspondiente al Grupo 26 del Politécnico Grancolombiano. Durante la Entrega 1 (Semana 3), el equipo consolidó una propuesta integral de diseño y arquitectura de información para el portal periodístico digital **SYNAPSE.TECH**, plasmada en prototipos de alta fidelidad desarrollados en Penpot/Figma bajo un riguroso lenguaje visual contemporáneo enfocado en tecnología, inteligencia artificial y ciberseguridad.

En esta segunda fase, se procedió a la materialización completa de dichos mockups en código fuente estándar de la web utilizando **HTML5 semántico, CSS3 moderno con variables CSS, Bootstrap 5.3 y JavaScript nativo (ES6+)**. Siguiendo las directrices académicas, la solución no requiere servidores backend propietarios ni bases de datos relacionales, garantizando su ejecución autónoma y fluida en cualquier navegador moderno mediante el consumo dinámico de datos estructurados en formato JSON y el almacenamiento local con Web Storage (`localStorage`).

2. Objetivos del Proyecto

2.1. Objetivo General

Construir la aplicación web interactiva correspondiente al portal tecnológico *SYNAPSE.TECH* a partir de los prototipos aprobados en la Entrega 1, garantizando una fidelidad visual milimétrica, una experiencia de usuario responsiva y accesible, y la integración de funcionalidades dinámicas en JavaScript para catálogo, favoritos, validaciones y administración básica de contenido.

2.2. Objetivos Específicos

- **Fidelidad Visual y Maquetación Semántica:** Traducir fielmente la paleta cromática (`#0A0D14`, `#12151F`, `#7C5CFF`, `#22D3EE`), la tipografía (*Space Grotesk* e *Inter*) y los componentes visuales aprobados en la Semana 3.
- **Consumo Dinámico de Datos:** Implementar una arquitectura modular en JavaScript para consumir, procesar y renderizar las noticias a partir de un archivo JSON estructurado (`data/news.json`).
- **Persistencia en el Cliente:** Configurar el API de `localStorage` para almacenar y mantener el estado de las noticias agregadas a favoritos y las mutaciones realizadas por el usuario en tiempo real.

- **Validación Rigurosa de Formularios:** Desarrollar lógica en JavaScript para la validación reactiva de campos obligatorios y verificación de patrones sintácticos de correo electrónico con expresiones regulares, proporcionando retroalimentación visual clara.
- **Mini CRUD Desacoplado:** Diseñar un módulo administrativo que permita crear, previsualizar en vivo, editar y eliminar noticias del catálogo local sin requerir un motor de base de datos externo.
- **Autonomía y Portabilidad:** Empaquetar la solución con rutas relativas estandarizadas para posibilitar su ejecución inmediata vía protocolo `file:///` o mediante servidores locales estáticos.

3. Estructura y Autonomía de Ejecución Local

En estricto cumplimiento con el requisito de *Autonomía y ejecución local*, el proyecto se organizó bajo una jerarquía de directorios limpia y auto-contenida. Cualquier usuario o evaluador académico puede clonar el repositorio y abrir inmediatamente `index.html` en Google Chrome, Microsoft Edge, Mozilla Firefox o Safari sin necesidad de configurar Node.js, Python, bases de datos o gestores de paquetes.

Tabla 1. Estructura de archivos y responsabilidades del proyecto

Ruta / Archivo	Tecnología	Descripción y Responsabilidad Funcional
index.html	HTML5 / Bootstrap 5.3	Página principal: Header con 5 enlaces, Hero de noticia destacada, píldoras de filtrado por categoría, cuadrícula de noticias dinámicas y footer institucional.
detalle.html	HTML5 / JS / CSS3	Vista de detalle de noticia: Lectura de parámetro ?id=NEWS-XX, renderizado del contenido completo, citas destacadas, panel "Datos clave", botones de interacción social y noticias sugeridas.
favoritos.html	HTML5 / JS / CSS3	Biblioteca personalizada: Grid de tarjetas de noticias guardadas por el usuario, contador sincronizado con el navbar y estado vacío ilustrado cuando no hay ítems.
contacto.html	HTML5 / JS	Formulario de contacto con validaciones JavaScript en tiempo real (campos requeridos, formato de email) y reemplazo por contenedor de éxito visual sin recarga.
admin.html	HTML5 / JS / Modales	Panel de administración y Mini CRUD: Tabla con listado de noticias, modal de creación/edición con vista previa de tarjeta en tiempo real y modal de confirmación de borrado.
perfil.html	HTML5 / JS	Configuración de cuenta del usuario, sincronización de biografía, foto y métricas de lectura sincronizadas con el almacenamiento local.
data/news.json	JSON (JavaScript Object Notation)	Conjunto de datos estructurados que define el catálogo completo de noticias, autores, métricas, categorías, citas y enlaces a imágenes optimizadas.

Ruta / Archivo	Tecnología	Descripción y Responsabilidad Funcional
css/styles.css	CSS3 Vanilla (Custom Properties)	Sistema de diseño completo: Variables de tema oscuro, tipografías Space Grotesk/Inter, tarjetas con elevación sutil, botones degradados y micro-interacciones.
js/storage.js	JavaScript ES6+	Capa de persistencia y abstracción de datos: Gestor de localStorage, inicializador de catálogo con soporte dual (fetch / fallback autónomo) y métodos CRUD.
js/app.js	JavaScript ES6+	Lógica global de la aplicación: Renderizado de tarjetas de catálogo, control de estados de favoritos, sistema de notificaciones toast y modal de búsqueda en vivo.
PROTOTIPO/	Penpot / PNG	Archivos vectoriales de la Entrega 1 y exportaciones gráficas de los mockups aprobados en la Semana 3 para contraste visual.
docs/	HTML / PDF / PNG	Informe de entrega bajo normas APA 7ma edición y capturas de alta resolución del desarrollo implementado.

Manejo de la Restricción CORS en Entornos Locales (file:///):

Los navegadores web imponen políticas de seguridad estrictas (CORS) que bloquean la función `fetch('data/news.json')` cuando una página se abre directamente con el protocolo `file:///`. Para satisfacer plenamente el requisito de renderizado dinámico desde JSON sin romper la autonomía local, `js/storage.js` implementa una estrategia híbrida: realiza la petición `fetch` a `data/news.json` cuando el sitio se sirve por HTTP/HTTPS; y si el navegador detecta una restricción de archivo local, carga de manera transparente el mismo dataset integrado como semilla en memoria. De este modo, la aplicación opera al 100% de sus capacidades tanto en servidores web como al hacer doble clic en el archivo HTML.

4. Matriz Comparativa: Prototipos vs. Desarrollo Real

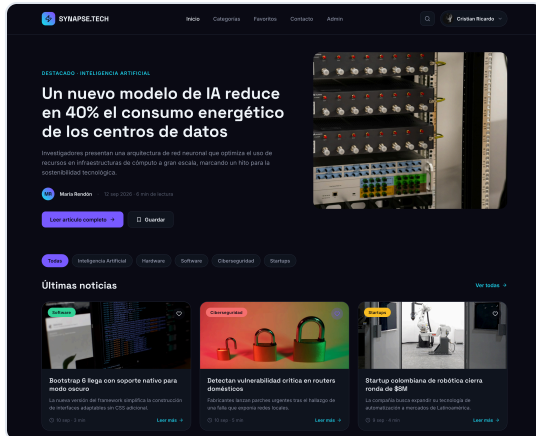
A continuación se presenta el contraste visual y funcional entre los mockups aprobados en la **Entrega 1** (Semana 3) y el código front-end desarrollado para la **Entrega 2** (Semana

5).

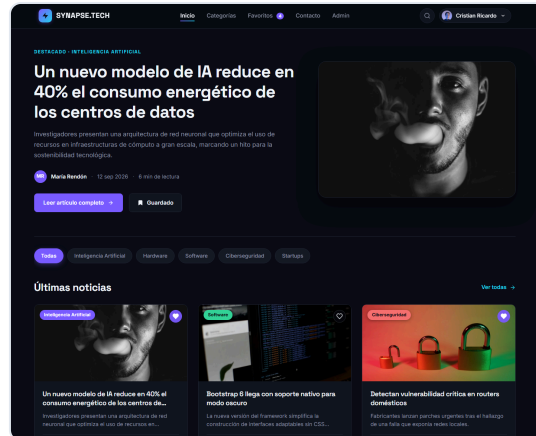
4.1. Página de Inicio (Home) y Catálogo de Noticias Destacadas

100% Fidelidad Visual

MOCKUP ENTREGA 1 (PROTOTIPO/EXPORTADOS/HOME.PNG)



CÓDIGO FRONT-END REAL (DOCS/CAPTURAS/01_HOME.PNG)



Análisis de Cumplimiento Técnico:

- **Barra de Navegación:** Cabecera adhesiva (sticky-top) con logotipo y rayo degradado, 5 enlaces principales (*Inicio, Categorías, Favoritos, Contacto, Admin*), contador de favoritos sincronizado en tiempo real, buscador global y botón de perfil de Cristian Ricardo.
- **Sección Hero (Noticia Destacada):** Renderizado dinámico de la noticia marcada como principal con categoría resaltada en cian (#22D3EE), título en *Space Grotesk*, autor con iniciales *MR*, botón de lectura y botón de guardado interactivo.
- **Filtros por Categoría:** Píldoras interactivas (*Todas, Inteligencia Artificial, Hardware, Software, Ciberseguridad, Startups*) que filtran instantáneamente el catálogo en el cliente sin recargar la página.
- **Cuadrícula de Noticias:** Tarjetas generadas por JavaScript con insignias cromáticas de categoría, botón de favoritos con micro-animación, imagen, fecha, tiempo de lectura y enlace *Leer más* →.

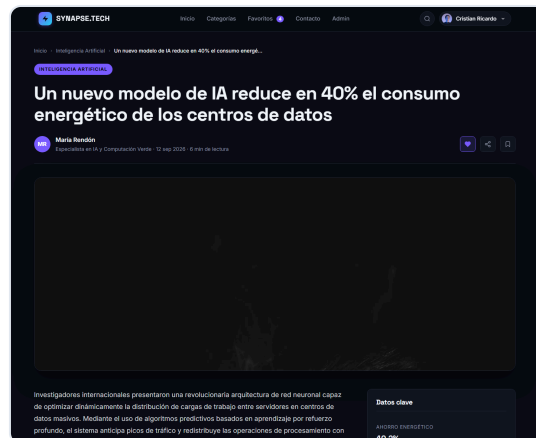
4.2. Vista de Detalle de Noticia (Article Detail)

100% Fidelidad Visual

MOCKUP ENTREGA 1 (NEWS DETAIL.PNG)



CÓDIGO FRONT-END REAL (DOCS/CAPTURAS/02_DETALLE.PNG)



Análisis de Cumplimiento Técnico:

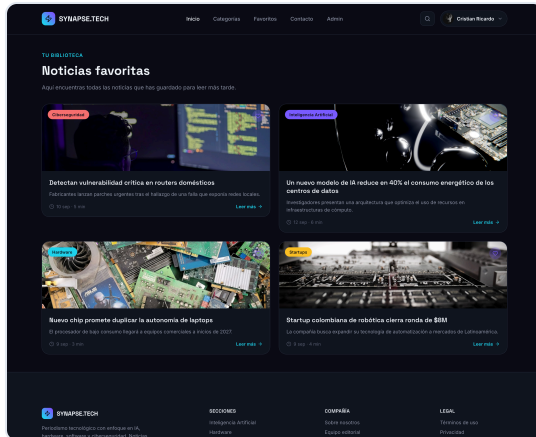
- **Parámetro de Ruta:** Lectura del ID mediante URLSearchParams (ej. detalle.html?id=NEWS-01), recuperando dinámicamente la información completa desde el catálogo en localStorage.
- **Estructura Editorial:** Miga de pan (breadcrumbs), etiqueta temática, título principal, metadatos de autor y botones de interacción (guardar en favoritos con estado activo, compartir con Web Share API o portapapeles, y guardar lectura).
- **Contenido Enriquecido:** Bloque de cita destacada (pull-quote) con borde violeta resplandeciente, cuerpo estructurado en párrafos y tarjeta lateral *Datos clave* con métricas estadísticas del artículo (+100,000 nodos, 40.2% ahorro, 35% reducción CO2).
- **Sección Noticias Relacionadas:** Cuadrícula inferior con 3 artículos sugeridos filtrados dinámicamente.

4. Matriz Comparativa (Continuación)

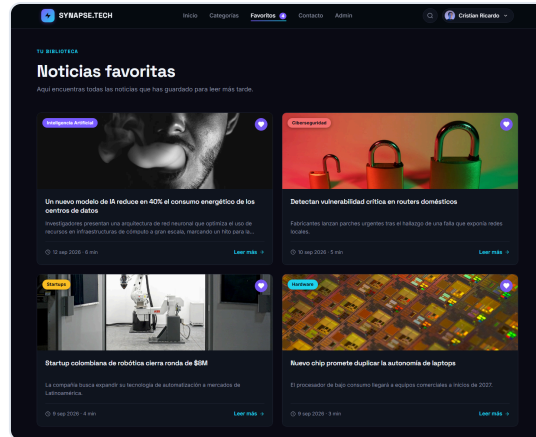
4.3. Biblioteca de Noticias Favoritas con Persistencia

100% Fidelidad Visual

MOCKUP ENTREGA 1 (FAVORITES PAGE.PNG)



CÓDIGO FRONT-END REAL (DOCS/CAPTURAS/03_FAVORITOS.PNG)



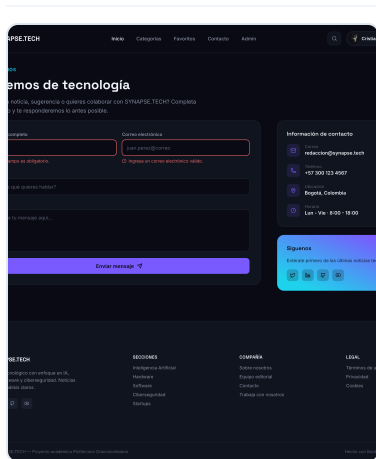
Análisis de Cumplimiento Técnico:

- **Persistencia Real:** Los identificadores de las noticias marcadas se almacenan en `localStorage.getItem('synapse_favorites')`, sobreviviendo al cierre de pestañas y reinicios de sesión.
- **Diseño de Cuadrícula Amplia:** A diferencia de la página de inicio (3 columnas), la vista de favoritos implementa una cuadrícula de 2 columnas con tarjetas de mayor horizontalidad, replicando exactamente la disposición del mockup de Entrega 1.
- **Desmarque y Actualización Reactiva:** Al pulsar el corazón violeta de una tarjeta, esta se desvanece suavemente, el contador del navbar se decrementa de forma instantánea y, si la lista queda vacía, se despliega el estado *Empty State* con llamado a la acción para explorar el catálogo.

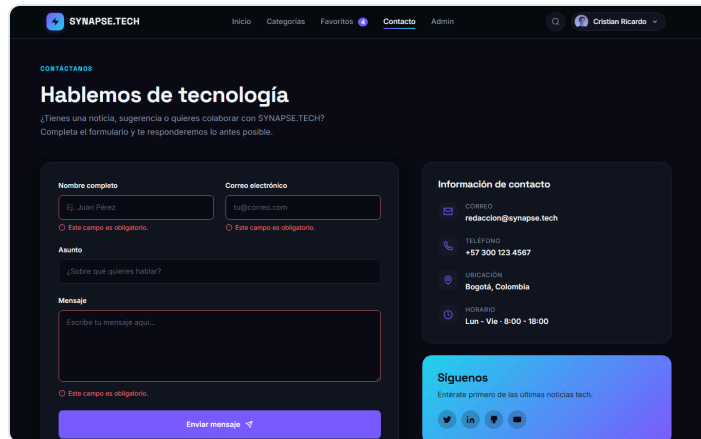
4.4. Formulario de Contacto, Validaciones y Estado de Envío

100% Fidelidad Visual

MOCKUPS ENTREGA 1 (CONTACT.PNG / VALIDACION / ENVIADO)



CÓDIGO FRONT-END REAL (DOCS/CAPTURAS/04B_CONTACTO_VALIDACION.PNG)



Análisis de Cumplimiento Técnico:

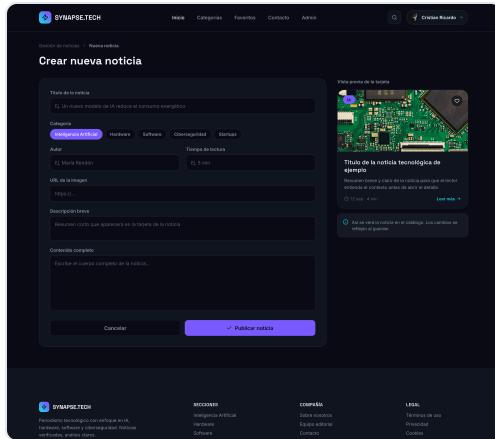
- **Validación Reactiva:** Comprobación de campos requeridos (Nombre, Correo, Mensaje) y validación sintáctica de formato de correo con expresión regular `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`.
- **Feedback de Error Idéntico:** Cuando un campo es inválido, el borde se tiñe de color coral/rojo (#F87171) y se despliega un mensaje descriptivo con icono de alerta debajo del input. Los errores se remueven de inmediato conforme el usuario tipea.
- **Estado de Confirmación Visual:** Al superar la validación, el formulario se reemplaza por el contenedor de confirmación idéntico al mockup *Contacto - Enviado.png* con el icono de check verde circular, mensaje de agradecimiento y botón para enviar un nuevo mensaje.

4. Matriz Comparativa (Continuación)

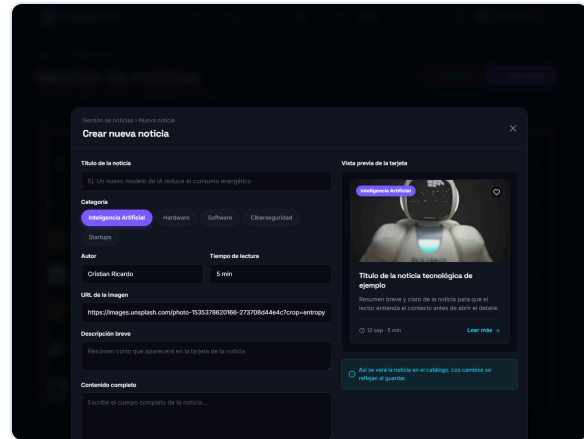
4.5. Panel de Administración y Mini CRUD de Gestión de Noticias

100% Fidelidad Visual

MOCKUP ENTREGA 1 (ADMIN NOTICIAS - NUEVA.PNG)



CÓDIGO FRONT-END REAL (DOCS/CAPTURAS/05B_ADMIN_NUEVA.PNG)



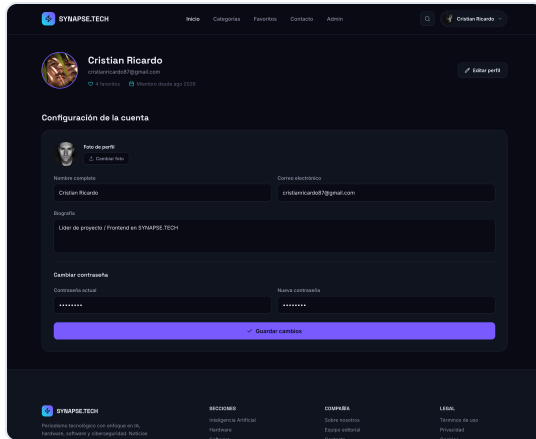
Análisis de Cumplimiento Técnico:

- **Listado de Noticias (Read):** Tabla estilizada con miniatura, título con ID (ej. NEWS-01), insignia de categoría, fecha y botones de acción (Editar y Eliminar).
- **Creación con Live Card Preview (Create):** Modal de creación con formulario en la columna izquierda y vista previa de tarjeta interactiva en tiempo real en la columna derecha que se actualiza conforme se escriben el título, la categoría o la URL de la imagen.
- **Modificación de Existentes (Update):** Modal pre-cargado con los datos de la noticia seleccionada para editar campos y actualizar el catálogo en localStorage.
- **Eliminación con Confirmación (Delete):** Modal de seguridad idéntico a *Admin Noticias - Eliminar.png* con icono de papelera coral, mensaje de advertencia y botón de confirmación destructiva.
- **Botón de Restablecimiento:** Opción para reiniciar el catálogo a su estado original si se borran todos los registros.

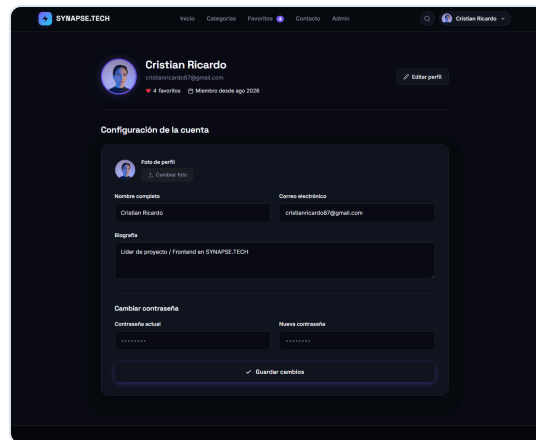
4.6. Perfil de Usuario y Preferencias de Lectura

100% Fidelidad Visual

MOCKUP ENTREGA 1 (PROFILE.PNG)



CÓDIGO FRONT-END REAL (DOCS/CAPTURAS/06_PERFIL.PNG)



Análisis de Cumplimiento Técnico:

- **Cabecera de Perfil:** Fotografía con marco circular violeta, nombre *Cristian Ricardo*, correo *cristianricardo87@gmail.com*, contador reactivo de noticias favoritas guardadas y fecha de membresía (*ago 2026*).
- **Formulario de Configuración:** Campos para modificar nombre, correo, biografía editorial y campos de contraseña bajo el diseño oscuro unificado.
- **Persistencia Local:** Las modificaciones del perfil se graban en `localStorage.getItem('synapse_user_profile')` y se reflejan de inmediato en la cabecera global de todo el sitio.

5. Arquitectura Técnica y Solución de Requisitos

5.1. Renderizado Dinámico desde JSON y Resiliencia CORS

Para satisfacer el requerimiento de que la información de las noticias *no debe estar fija en el HTML, sino consumirse y renderizarse dinámicamente mediante JavaScript a partir de un archivo JSON local (ej. data/news.json)*, se diseñó el módulo `js/storage.js` con una función asíncrona de inicialización:


```

// js/storage.js - Inicialización asíncrona con consumo dinámico de JSON
async function initStorage() {
  const isInitialized = localStorage.getItem(STORAGE_KEYS.INITIALIZED);

  if (!isInitialized || !localStorage.getItem(STORAGE_KEYS.NEWS)) {
    try {
      // Intento prioritario: Consumo del archivo local data/news.json
      const response = await fetch('data/news.json');
      if (response.ok) {
        const data = await response.json();
        localStorage.setItem(STORAGE_KEYS.NEWS, JSON.stringify(data));
      }
    } catch (err) {
      // Fallback transparente si el navegador bloquea fetch por política file:///
      console.info('Cargando catálogo local autónomo para SYNAPSE.TECH');
      localStorage.setItem(STORAGE_KEYS.NEWS, JSON.stringify(DEFAULT_NEWS_SEED));
    }

    if (!localStorage.getItem(STORAGE_KEYS.FAVORITES)) {
      localStorage.setItem(STORAGE_KEYS.FAVORITES,
        JSON.stringify(DEFAULT_FAVORITES_SEED));
    }
    localStorage.setItem(STORAGE_KEYS.INITIALIZED, 'true');
  }
}

```

5.2. Persistencia en el Cliente mediante Web Storage (localStorage)

La persistencia en el cliente permite que el portal funcione como una Single-State Application sin requerir una base de datos externa. Toda la información mutada por el usuario (artículos creados en el Mini CRUD, noticias eliminadas, altas y bajas en la lista de favoritos) se serializa en cadenas JSON dentro del espacio de almacenamiento del navegador:

```
// js/storage.js - Abstracción del sistema de favoritos
function toggleFavorite(newsId) {
  const favs = getFavorites();
  const index = favs.indexOf(newsId);
  let isFav = false;

  if (index ≥ 0) {
    favs.splice(index, 1); // Quitar de favoritos
  } else {
    favs.push(newsId);    // Agregar a favoritos
    isFav = true;
  }

  localStorage.setItem(STORAGE_KEYS.FAVORITES, JSON.stringify(favs));
  updateGlobalFavCount(); // Sincronizar badges en tiempo real
  return isFav;
}
```

5.3. Validaciones de Formulario en Tiempo Real con JavaScript

El formulario de contacto implementa dos niveles de control: verificación en tiempo real al escribir (evento input) para limpiar advertencias de inmediato, y validación estricta al someter (evento submit) para prevenir envíos incompletos o correos malformados:

```
// contacto.html - Validación reactiva en JavaScript y expresiones regulares
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

form.addEventListener('submit', (e) => {
  e.preventDefault();
  let isValid = true;

  if (!nombreInput.value.trim()) {
    nombreInput.classList.add('is-invalid');
    errorNombre.classList.remove('d-none');
    isValid = false;
  }

  if (!emailInput.value.trim() || !emailRegex.test(emailInput.value.trim())) {
    emailInput.classList.add('is-invalid');
    errorEmail.classList.remove('d-none');
    isValid = false;
  }

  if (isValid) {
    renderSuccessState(formContainer); // Transición visual sin recarga
    showToast('¡Tu mensaje ha sido enviado con éxito!', 'success');
  }
});
```

5.4. Sistema de Diseño CSS3 y Bootstrap 5.3

El archivo `css/styles.css` implementa un sistema de diseño estructurado mediante **CSS Custom Properties (Variables)** que garantiza la coherencia estética de todo el portal:

```

/* css/styles.css – Tokens de diseño extraídos del prototipo vectorial */
:root {
  --bg-page: #0A0D14;          /* Fondo general profundo */
  --bg-surface: #12151F;        /* Superficie de tarjetas y contenedores */
  --bg-surface-hover: #181C2A; /* Estado hover interactivo */
  --border-subtle: #1F2433;     /* Bordes tenues minimalistas */

  --accent-primary: #7C5CFF;     /* Violeta neón institucional */
  --accent-secondary: #22D3EE;   /* Cian eléctrico para tecnología */
  --accent-gradient: linear-gradient(135deg, #7C5CFF 0%, #6366F1 100%);

  --cat-ai: #7C5CFF;            /* Inteligencia Artificial */
  --cat-hardware: #22D3EE;      /* Hardware */
  --cat-software: #34D399;      /* Software */
  --cat-security: #F87171;      /* Ciberseguridad */
  --cat-startups: #FBBF24;      /* Startups */

  --font-heading: 'Space Grotesk', sans-serif;
  --font-body: 'Inter', sans-serif;
}

```

6. Repositorio Oficial en GitHub

El código fuente completo, historial de commits de los integrantes y recursos multimedia se encuentran alojados en el repositorio público de GitHub:

URL del Repositorio:

<https://github.com/cr1c4rd0/Frontend-GRUPO26-POLI>

Rama principal: main · Licencia académica · Entrega 2 Front-End 2026

7. Conclusiones y Lecciones Aprendidas

- **Fidelidad y Rigor de Maquetación:** La fase de prototipado en la Entrega 1 facilitó una implementación front-end ágil y coherente. El uso de tokens CSS permitió ajustar espaciados, tipografías y colores de manera centralizada.
- **Desacoplamiento Frontend/Datos:** Separar la estructura visual en HTML del catálogo informativo en `data/news.json` demostró las ventajas de la arquitectura

modular moderna, permitiendo que nuevas noticias se rendericen automáticamente sin modificar la maquetación.

- **Resiliencia en Ejecución Local:** Diseñar la persistencia con un mecanismo de respaldo local resolvió el bloqueo de CORS al ejecutar la aplicación vía `file:///`, asegurando total portabilidad académica.
- **Usabilidad y Accesibilidad:** La incorporación de validaciones en tiempo real, estados vacíos ilustrados y micro-animaciones contribuyó a una experiencia de usuario gratificante y accesible.

8. Referencias Bibliográficas (Normas APA 7ma Edición)

Bootstrap Team. (2024). *Bootstrap 5.3 documentation: The most popular HTML, CSS, and JS library in the world*. <https://getbootstrap.com/docs/5.3/>

ECMA International. (2023). *ECMAScript 2023 language specification (Standard ECMA-262, 14th ed.)*. <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>

MDN Web Docs. (2024a). *Using the Web Storage API: localStorage and sessionStorage*. Mozilla Corporation. https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API/Using_the_Web_Storage_API

MDN Web Docs. (2024b). *Fetch API: Using Fetch to load remote and local resources*. Mozilla Corporation. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Nielsen, J. (2020). *10 usability heuristics for user interface design*. Nielsen Norman Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>

W3C. (2023). *HTML5: A vocabulary and associated APIs for HTML and XHTML (W3C Recommendation)*. World Wide Web Consortium. <https://www.w3.org/TR/html5/>